

Lesson4_3_5

September 5, 2026

1 Introduction

This web site is running a “Jupyter notebook,” which allows you to execute interactive Octave code directly in your browser. Each (part of a code) line preceded by a percent symbol “%” is a code comment.

To execute any Octave commands, first enter the commands in a code cell, and then depress the “shift” and “enter” keys on your keyboard at the same time, making sure that your cursor is placed inside of the code cell. I refer to this as “< shift > < enter >” in my instructions below.

The output from this notebook will be used to provide answers for this lesson’s practice quiz. As you proceed through the specialization, you will use more and more Jupyter notebooks to write Octave code to implement Kalman-filter algorithms.

```
[1]: % Place your cursor in this input code cell. Then, type < shift > < enter > to
      ↪ execute

rms = @(x) sqrt(mean((x).^2)); % define root-mean-squared function

% The car has already been simulated offline and the data are stored in
      ↪ "observations.mat".
% We load that datafile here.
load readonly/observations.mat x z
nIter = length(x);

% Define some constants related to the problem
sigmaV = 5*pi/180;
sigmaW = 10*pi/180;
meanX0 = 50*pi/180;
sigmaX0 = 10*pi/180;
omega = 2*pi/180;
xo = 500;
yo = 500;
r = 200;
```

1.0.1 Implement the particle-filter solution

The next notebook cells implement the particle-filter SIS method.

```

[2]: % Create the particles
nPart = 500;
xp = zeros(nPart,nIter);
wp = zeros(nPart,nIter);
xHatPart = zeros(nIter,1);

% Set initial random-number-generator state to make simulation
% results repeatable. Note that Octave's random-number generator
% is not the same as MATLAB's, so the results you compute here
% will not be identical to those in the lesson notes.
randn("state",1);

% Initialize particle values and weights
xp(:,1) = meanX0 + sigmaX0*randn(nPart,1);
wp(:,1) = 1/nPart;

% Implement SIS for the rest of the trajectory
for k=2:nIter
    for cp=1:nPart
        % Draw new particle from  $q(x\{k\}/x\{k-1\},z\{k\}) = f(x\{k\}/x\{k-1\})$ 
        xp(cp,k) = xp(cp,k-1) + omega + sigmaW*randn(1);
        % Compute the expected value of the measurement given  $x\{k\}$ 
        zp = atan((r*sin(xp(cp,k))+yo)./(r*cos(xp(cp,k))+xo));
        % Compute  $f(z\{k\}/x\{k\})$ 
        likelihood = normpdf(z(k),zp,sigmaV);
        % Compute unnormalized weight
        %  $wptilde(cp,k) = f(x\{k\}/x\{k-1\}) / q(x\{k\}/x\{k-1\},z\{k\}) * f(z\{k\}/x\{k\}) * wp(cp,k-1)$ 
        wp(cp,k) = wp(cp,k-1)*likelihood;
    end
    wp(:,k) = wp(:,k)/sum(wp(:,k)); % Normalize weights
    xHatPart(k) = sum(wp(:,k).*xp(:,k)); % Compute mean based on particles

    if mod(k,10)==0
        fprintf('Completed iteration %d out of %d\n',k,nIter);
    end
end
fprintf('Done!\n');

```

```

Completed iteration 10 out of 101
Completed iteration 20 out of 101
Completed iteration 30 out of 101
Completed iteration 40 out of 101
Completed iteration 50 out of 101
Completed iteration 60 out of 101
Completed iteration 70 out of 101
Completed iteration 80 out of 101

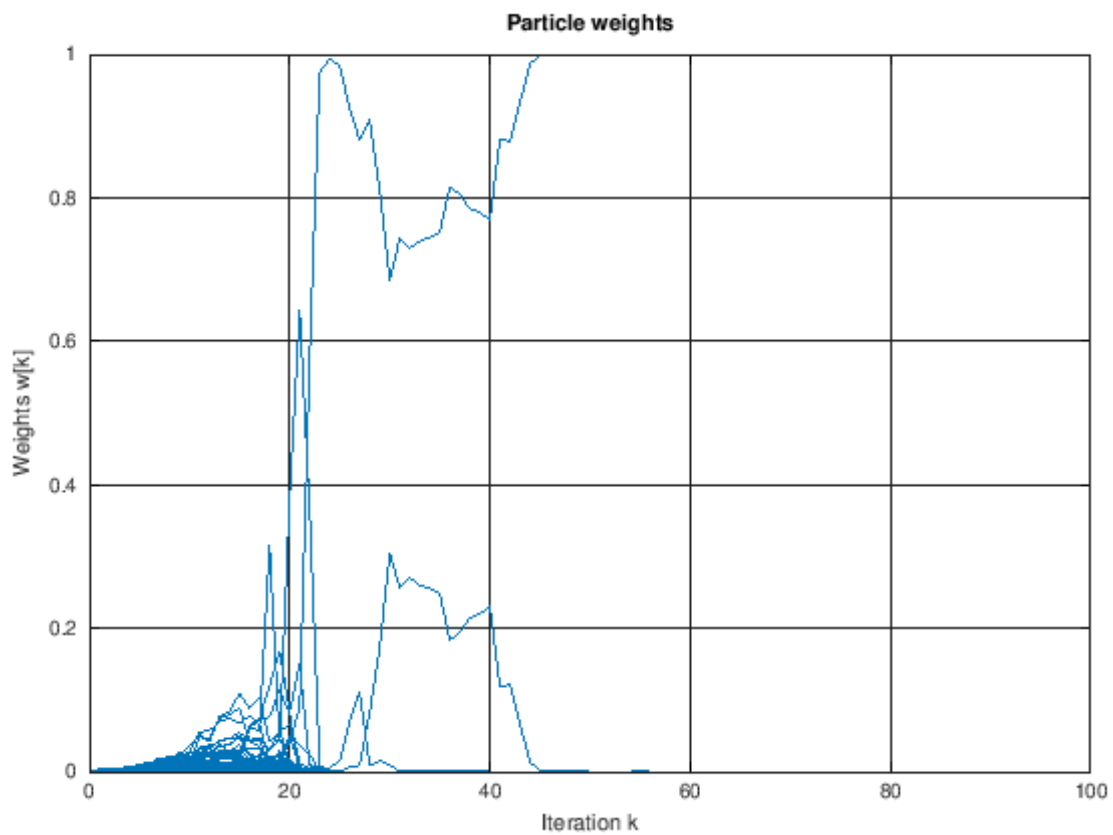
```

Completed iteration 90 out of 101
Completed iteration 100 out of 101
Done!

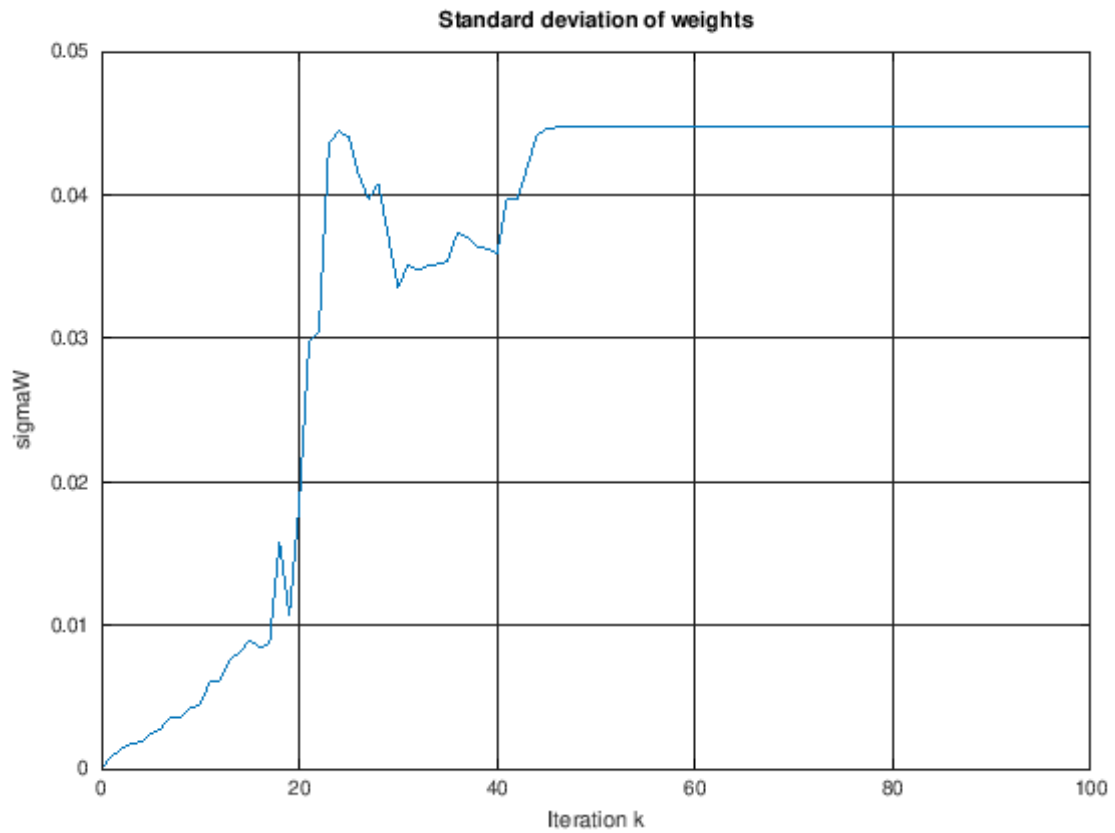
1.0.2 Visualize particle weights

Trying to understand the problem with the basic SIS algorithm; visualize particle weights.

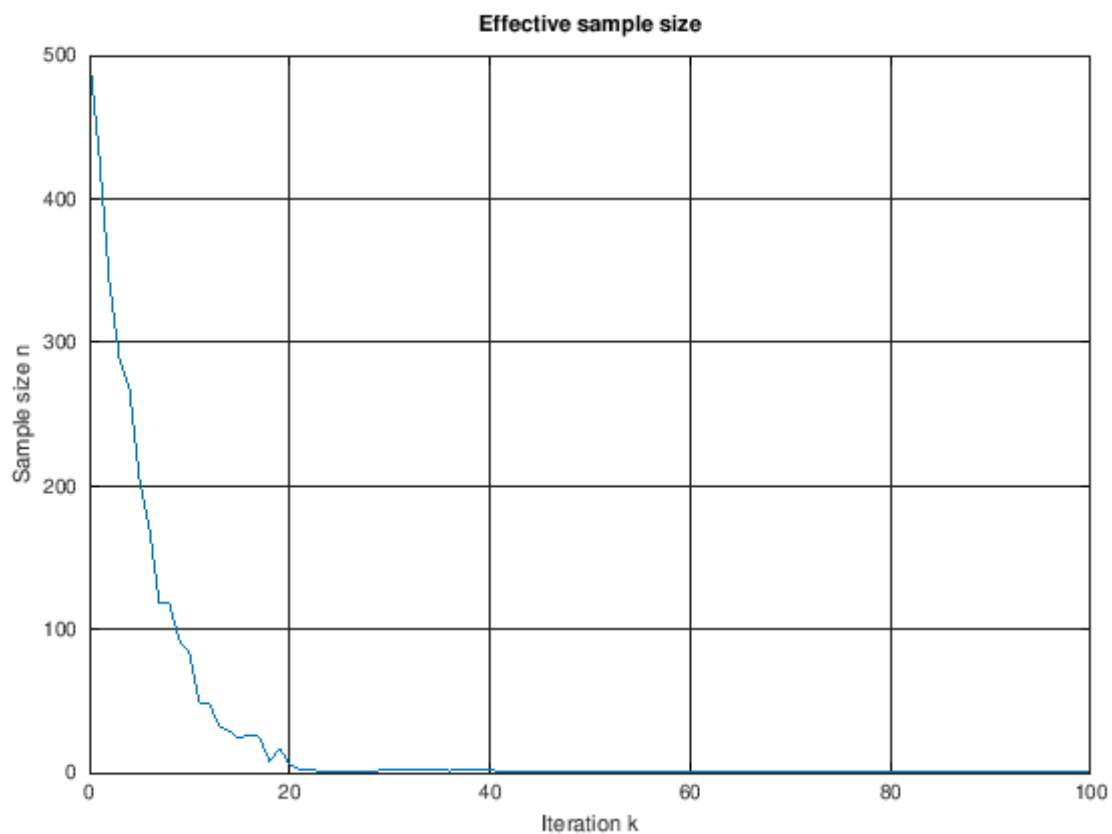
```
[3]: % Plot the particle weights versus iteration
CL = lines;
plot(0:nIter-1,wp,'color',CL(1,:)); grid on
xlabel('Iteration k');
ylabel('Weights w[k]'); title('Particle weights');
xlim([0 nIter-1]); ylim([0 1]);
```



```
[4]: % Plot the standard deviation of the weights versus iteration
plot(0:nIter-1,std(wp)); grid on
xlabel('Iteration k'); ylabel('sigmaW');
title('Standard deviation of weights'); xlim([0 nIter-1]);
```



```
[5]: % Plot the effective sample size
nEff = 1./sum(wp.^2);
h = plot(0:nIter-1,nEff); grid on
xlabel('Iteration k');
ylabel('Sample size n'); title('Effective sample size');
xlim([0 nIter-1]); ylim([0 nPart]);
```



[]: